

Projeto 1 de Estrutura de Dados II

Compressão de Arquivos com o Algoritmo de Huffman

Prof. Dr. Jean M. Laine

Conteúdo

1	Introdução	2
2	Objetivos do Projeto	2
3	Descrição Detalhada do Algoritmo	3
3.1	Parte 1: O Processo de Compressão	3
3.1.1	Passo 1: Análise de Frequência	3
3.1.2	Passo 2: A Fila de Prioridades (Min-Heap)	3
3.1.3	Passo 3: Construção da Árvore de Huffman	4
3.1.4	Passo 4: Geração da Tabela de Códigos	4
3.1.5	Passo 5: Codificação e Escrita do Arquivo	4
3.2	Parte 2: O Processo de Descompressão	4
3.3	Parte 3: O Percurso na Descompressão	4
4	Exemplo Completo Passo a Passo	5
4.1	Passo 1: Análise de Frequência	5
4.2	Passo 2: Construção da Árvore via Min-Heap	5
4.3	Passo 3: Árvore de Huffman Final	5
4.4	Passo 4: Tabela de Códigos Resultante	6
4.5	Passo 5: Codificação	6
4.6	Passo 6: Descompressão (Usando o Percurso Guiado)	6
5	Saída Esperada do Programa	7
6	Requisitos de Implementação, Entrega e Observações	8
7	Análises Requeridas	8
8	Critérios de Avaliação	9
9	Referências e Material de Apoio	9

1 Introdução

O Algoritmo de Huffman é um método de compressão sem perdas que atribui códigos de tamanho variável aos caracteres de entrada com base em suas frequências. Caracteres frequentes recebem códigos curtos e caracteres raros recebem códigos mais longos, otimizando o tamanho final do arquivo.

Embora seja um algoritmo clássico, sua relevância persiste até hoje, sendo um componente fundamental em diversas tecnologias que usamos diariamente. Entender o seu funcionamento é compreender uma das bases da transmissão e armazenamento eficiente de dados. Algumas de suas aplicações mais notáveis incluem:

- **Compressão de Arquivos (.zip, .gzip):** O popular algoritmo DEFLATE, que é o coração dos formatos ZIP e GZIP, utiliza uma combinação do algoritmo LZ77 com a codificação de Huffman. O Huffman é a etapa final responsável por comprimir de forma eficiente os dados já processados pelo LZ77.
- **Formatos de Imagem (JPEG, PNG):** No formato JPEG, após as transformações matemáticas que processam as cores da imagem, a codificação de Huffman é usada para comprimir os coeficientes resultantes, reduzindo drasticamente o tamanho do arquivo. No formato PNG, o Huffman também é um componente essencial do seu método de compressão sem perdas.
- **Formatos de Áudio e Vídeo (MP3, MPEG):** Padrões de compressão de mídia como MP3 e MPEG usam variações da codificação de Huffman como uma de suas etapas finais para representar os dados de áudio e vídeo de forma compacta.
- **Protocolos de Rede (HTTP/2):** Para tornar a navegação na web mais rápida, o protocolo HTTP/2 usa o HPACK para compressão de cabeçalhos. Uma das estratégias do HPACK é usar uma tabela de Huffman estática para representar strings comuns nos cabeçalhos HTTP, reduzindo a quantidade de dados transmitidos entre o navegador e o servidor.

Portanto, ao implementar o algoritmo de Huffman e desenvolver este projeto, vocês não estarão apenas praticando conceitos de estruturas de dados e os usando para resolver problemas, mas também criando uma peça de tecnologia que é a base para a transmissão e o armazenamento eficiente de dados na computação moderna.

2 Objetivos do Projeto

- Implementar e utilizar uma Fila de Prioridades (Min-Heap).
- Modelar, construir e percorrer uma Árvore Binária (a Árvore de Huffman).
- Desenvolver uma aplicação de linha de comando para compressão e descompressão.
- Gerar um novo arquivo com os dados comprimidos e ser capaz de reconstruir o arquivo original a partir dele.
- Manipular arquivos em nível de leitura e escrita em Java.
- Praticar o trabalho em equipe (máximo 4 e mínimo 3 alunos por grupo) e a divisão de tarefas.

3 Descrição Detalhada do Algoritmo

O projeto é dividido em funcionalidades principais. Uma demonstração completa com um exemplo prático é detalhada na Seção 4.

3.1 Parte 1: O Processo de Compressão

3.1.1 Passo 1: Análise de Frequência

Leia o arquivo de entrada e conte a frequência de cada caractere. Para isso, **você deve utilizar um vetor de inteiros (`int[]`) de tamanho 256** (para cobrir a tabela ASCII).

3.1.2 Passo 2: A Fila de Prioridades (Min-Heap)

Você deverá implementar uma Fila de Prioridades usando um **Min-Heap**.

Nota Conceitual Importante: É crucial entender a diferença entre a **ideia** de um Min-Heap e sua **implementação**.

- **Conceitualmente:** Um Min-Heap é uma **árvore binária completa**.
- **Na Prática:** Representamos essa árvore usando um **vetor** ou **ArrayList**. Para um nó no índice i , seu filho esquerdo está em $2*i + 1$, o direito em $2*i + 2$, e o pai em $(i - 1) / 2$.

Sua classe `MinHeap` conterá um `ArrayList<No>`.

Estrutura do Nó: A classe `No` será usada para construir a **Árvore de Huffman**.

```
class No implements Comparable<No> {
    char caractere;
    int frequencia;
    No esquerda, direita;
    // Implementar construtor e o metodo compareTo
    @Override
    public int compareTo(No outroNo) {
        return this.frequencia - outroNo.frequencia;
    }
}
```

Analisando a Classe No: Esta classe é a base para as duas estruturas do projeto.

- `char caractere`, `int frequencia`: Guardam os dados essenciais de cada folha da árvore: o caractere em si e quantas vezes ele apareceu.
- `No esquerda`, `No direita`: São as referências (ponteiros) usadas para construir a **Árvore de Huffman** final. Note que estes atributos pertencem à árvore explícita que será gerada, e não à estrutura do Min-Heap (que é implícita, em um vetor).
- `implements Comparable<No>`: É o "contrato" que a classe faz ao Java, garantindo que objetos do tipo `No` sabem como se comparar e, portanto, podem ser ordenados.

- `compareTo(No outroNo)`: É a implementação daquele contrato. Este método é o "cérebro" da comparação, sendo invocado pelo Min-Heap para decidir qual nó tem a menor frequência (e, portanto, maior prioridade).

3.1.3 Passo 3: Construção da Árvore de Huffman

Enquanto o Min-Heap tiver mais de um nó, remova os dois de menor frequência, combine-os em um novo nó interno e insira este novo nó de volta no heap.

3.1.4 Passo 4: Geração da Tabela de Códigos

Com a árvore completa, percorra-a recursivamente. Para armazenar a tabela, **utilize um vetor de Strings** (`String[]`) **de tamanho 256**.

3.1.5 Passo 5: Codificação e Escrita do Arquivo

Ao final do processo, o programa deve **gerar um novo arquivo de saída** (ex: com extensão `.huff`). Este arquivo deve conter um **cabeçalho** (com a tabela de frequências) e os **dados comprimidos**.

3.2 Parte 2: O Processo de Descompressão

O processo de descompressão consiste em reconstruir a árvore a partir do cabeçalho de um arquivo previamente comprimido e, em seguida, usar essa árvore para decodificar o fluxo de bits, gerando o arquivo original.

3.3 Parte 3: O Percurso na Descompressão

A descompressão utiliza um método de percurso, mas de uma forma diferente dos percursos completos como Pré-Ordem ou Em-Ordem. Em vez de visitar todos os nós da árvore de uma só vez, a descompressão realiza um **"percurso guiado pelos dados"**.

Funciona da seguinte forma:

1. Comece na raiz da árvore de Huffman.
2. Leia um bit do arquivo comprimido. Se for '0', desça para o filho da esquerda. Se for '1', desça para a direita.
3. Verifique se o nó atual é uma folha.
 - Se **não for** uma folha, repita o passo 2.
 - Se **for** uma folha, você decodificou um caractere! Escreva o caractere no arquivo de saída.
4. Após decodificar um caractere, **volte para a raiz** e repita todo o processo a partir do passo 1 para o próximo caractere.

Este ciclo de "percursos curtos" (da raiz até uma folha) continua até que todo o fluxo de bits do arquivo comprimido tenha sido consumido.

4 Exemplo Completo Passo a Passo

Vamos usar um exemplo simples. Imagine que seu arquivo de entrada (.txt), tenha uma única linha com a palavra "BANANA", apenas. Vamos usar este exemplo para ilustrar todo o processo.

4.1 Passo 1: Análise de Frequência

Após ler a string "BANANA", o vetor de frequências conterá os seguintes valores (outras posições são 0):

- frequencias['B'] = 1
- frequencias['A'] = 3
- frequencias['N'] = 2

4.2 Passo 2: Construção da Árvore via Min-Heap

Primeiro, criamos os nós folha e os inserimos no Min-Heap. O heap (representado como um vetor) é ordenado pela frequência.

Estado Inicial do Heap:

Vetor do Heap: [No('B',1), No('N',2), No('A',3)]

Iteração 1: Removemos No('B',1) e No('N',2), criamos um nó pai N1 com frequência 3 (1+2), e o inserimos de volta.

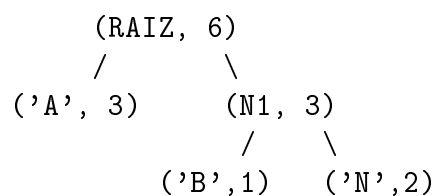
Estado do Heap após a Iteração 1:

Vetor do Heap: [No('A',3), N1(freq:3)]

Iteração 2: Removemos No('A',3) e N1(freq:3), criamos o nó RAIZ com frequência 6 (3+3), e o inserimos de volta. O processo termina.

4.3 Passo 3: Árvore de Huffman Final

A estrutura da árvore resultante é:



4.4 Passo 4: Tabela de Códigos Resultante

Percorrendo a árvore (esquerda='0', direita='1'):

- A: '0'
- B: '10'
- N: '11'

4.5 Passo 5: Codificação

"BANANA-> 100110110 (9 bits). **Comparação:** Original (ASCII) = 6 caracteres * 8 bits = 48 bits. Economia significativa.

4.6 Passo 6: Descompressão (Usando o Percurso Guiado)

Para decodificar 100110110:

- Começa na raiz. Lê 10 -> Chega na folha 'B'. Escreve B. Volta para a raiz.
- Lê 0 -> Chega na folha 'A'. Escreve A. Volta para a raiz.
- Lê 11 -> Chega na folha 'N'. Escreve N.
- ...e assim por diante até reconstruir "BANANA".

5 Saída Esperada do Programa

Para facilitar a correção e a depuração, seu programa deve imprimir as estruturas de dados intermediárias no console (`System.out`) durante o processo de **compressão**. A impressão deve seguir o formato do exemplo abaixo.

Exemplo de Saída para o arquivo com "BANANA":

```
-----  
ETAPA 1: Tabela de Frequencia de Caracteres  
-----
```

```
Caractere 'B' (ASCII: 66): 1  
Caractere 'A' (ASCII: 65): 3  
Caractere 'N' (ASCII: 78): 2
```

```
-----  
ETAPA 2: Min-Heap Inicial (Vetor)  
-----
```

```
[ No('B',1), No('N',2), No('A',3) ]
```

```
-----  
ETAPA 3: Arvore de Huffman  
-----
```

```
- (RAIZ, 6)  
  - ('A', 3)  
  - (N1, 3)  
    - ('B', 1)  
    - ('N', 2)
```

```
-----  
ETAPA 4: Tabela de Codigos de Huffman  
-----
```

```
Caractere 'A': 0  
Caractere 'B': 10  
Caractere 'N': 11
```

```
-----  
ETAPA 5: Resumo da Compressao  
-----
```

```
Tamanho original....: 48 bits (6 bytes)  
Tamanho comprimido...: 9 bits (2 bytes)  
Taxa de compressao...: 81.25%  
-----
```

6 Requisitos de Implementação, Entrega e Observações

- **Linguagem:** O projeto deve ser implementado em **Java**. Adicione o nome completo dos integrantes do grupo no cabeçalho do(s) arquivo(s).
- **Operação de Arquivos:** O programa **nunca** deve modificar o arquivo de entrada original. A compressão deve sempre gerar um novo arquivo de saída, e a descompressão também.
- **Execução:** O programa que será testado deve ser executável via linha de comando e ter o nome huffman.jar. As execuções e testes deverão ser feitos através dos seguintes comandos:

```
# Para comprimir
java -jar huffman.jar -c <arquivo_original> <arquivo_comprimido>

# Para descomprimir
java -jar huffman.jar -d <arquivo_comprimido> <arquivo_restaurado>
```

- **Relatório:** Um relatório em PDF (mínimo de 3 e máximo de 6 páginas) explicando as decisões de projeto e a implementação das estruturas de dados. O relatório deve obrigatoriamente conter a seção de "Análises Requeridas", apresentando as tabelas/gráficos e as discussões sobre a performance e as taxas de compressão.
- **Entrega:** Código-fonte completo do projeto Java (.java), o arquivo huffman.jar e o relatório.
- **Apresentação:** Apresentar o projeto é obrigatório para a nota.

7 Análises Requeridas

O relatório final deve conter as seguintes análises experimentais sobre o seu programa. Para garantir uma base de comparação consistente entre todos os grupos, será fornecido um arquivo de texto padrão, chamado `arq_de_teste.txt`. As análises de performance e taxa de compressão para “texto comum” devem ser realizadas **utilizando este arquivo e documentadas no relatório**. As análises com os outros tipos de arquivo (código-fonte, etc.) continuam sendo importantes para uma avaliação mais ampla e completa.

Parte 1: Análise de Performance (Tempo):

- **O que fazer?** Meça o tempo de execução (em milissegundos) das suas funções de compressão e descompressão. Teste com arquivos de texto de diferentes tamanhos (ex: 1KB, 100KB, 1MB, 10MB). Para medir o tempo em Java, use ‘System.nanoTime()’.
- **O que documentar no relatório?** Uma seção contendo uma tabela ou gráfico mostrando a relação entre o tamanho do arquivo e o tempo de execução. Discuta se o crescimento do tempo é linear e se corresponde ao esperado pela análise de complexidade teórica do algoritmo.

Parte 2: Comparação de Taxas de Compressão (Espaço):

- **O que fazer?** Calcule a taxa de compressão para diferentes tipos de arquivo. A taxa pode ser calculada com a fórmula:

$$\text{Taxa de Compressão} = \left(1 - \frac{\text{Tamanho Comprimido}}{\text{Tamanho Original}} \right) \times 100\%$$

Teste com arquivos de texto de características variadas, como: um texto comum (capítulo de livro), um código-fonte (.java), um arquivo muito repetitivo (ex: "AA-AAA...") e um arquivo com caracteres aleatórios.

- **O que documentar no relatório?** Uma tabela mostrando a taxa de compressão para cada tipo de arquivo testado. O mais importante é a sua **análise**: explique por que as taxas variaram. Por que o arquivo repetitivo comprimiu tão bem? Por que o arquivo aleatório teve uma compressão ruim ou até negativa (aumentou de tamanho)? Sua análise deve demonstrar que você entendeu quando e por que o algoritmo de Huffman é eficiente.

8 Critérios de Avaliação

A avaliação do projeto será baseada na rubrica detalhada na Tabela 1. A pontuação total é de 10,0 pontos, distribuídos entre os critérios de funcionalidade, implementação das estruturas de dados, e qualidade geral do código e do relatório.

9 Referências e Material de Apoio

- **Codificação de Huffman:**
<https://www.ime.usp.br/~pf/estruturas-de-dados/aulas/huffman.html>
- **GeeksforGeeks - Heap Data Structure:**
<https://www.geeksforgeeks.org/heap-data-structure/>
- **GeeksforGeeks - PriorityQueue em Java**
<https://www.geeksforgeeks.org/java/priority-queue-in-java/>
- **Livro - Algoritmos: Teoria e Prática**
CORMEN, Thomas H. et al. *Algoritmos: Teoria e Prática*. 3ª ed. Rio de Janeiro: LTC, 2012.

Tabela 1: Rubrica de Avaliação do Projeto

Categoria	Subitem	Critério de Avaliação	Pontos
1. Funcionalidade e Corretude (6,0)	1.1 Compilação e Execução	O programa compila e executa sem erros conforme a especificação.	1,0
	1.2 Compressão	Gera tabelas corretas (frequência, códigos), saída no console no formato correto e cria o arquivo de saída.	2,5
	1.3 Descompressão	O arquivo descomprimido é perfeitamente idêntico ao original; lida com erros de arquivo inválido.	2,5
2. Implementação das Estruturas (2,0)	2.1 Min-Heap	Implementado com vetor/ArrayList e com operações eficientes (logarítmicas).	1,0
	2.2 Árvore de Huffman	Classe No e algoritmo de construção da árvore estão corretos.	1,0
3. Qualidade e Relatório (2,0)	3.1 Qualidade do Código	Código bem organizado, comentado, com bons nomes de variáveis/métodos.	1,0
	3.2 Relatório e Análises	Relatório claro e bem estruturado, contendo as análises requeridas.	1,0
TOTAL			10,0